Занятие 48.1. Введение в std::array

На предыдущих уроках мы подробно говорили о **фиксированных** и **динамических** массивах. Хотя они очень полезны и активно используются в языке C++, у них также есть свои недостатки: фиксированные массивы **распадаются в указатели**, теряя информацию о своей длине; в динамических массивах проблемы могут возникнуть с освобождением памяти и с попытками изменить их длину после выделения.

Поэтому в Стандартную библиотеку C++ добавили функционал, который упрощает процесс управления массивами: std::array и std::vector. На этом уроке мы рассмотрим std::array, а на следующем — std::vector.

Оглавление:

1. **Введение в std::array**
2. **Размер и сортировка**
3. **Заключение**

Введение в std::array

Представленный в C++11, **std::array** — это фиксированный массив, который не распадается в указатель при передаче в функцию. std::array определяется в **заголовочном файле** array, внутри **пространства имен** std. Объявление переменной std::array следующее:

```
#include <array>

std::array<int, 4> myarray; // объявляем массив типа int длиной 4
```

Подобно обычным фиксированным массивам, длина std::array должна быть установлена во время компиляции. std::array можно инициализировать с использованием списка инициализаторов или **uniform-инициализации**:

```
std::array<int, 4> myarray = { 8, 6, 4, 1 }; // список инициализаторов
std::array<int, 4> myarray2 { 8, 6, 4, 1 }; // uniform-инициализация
```

В отличие от стандартных фиксированных массивов, в std::array вы не можете пропустить (не указывать) длину массива:

```
std::array<int, > myarray = { 8, 6, 4, 1 }; // нельзя, должна быть указана длина массива
```

Также можно присваивать значения массиву с помощью списка инициализаторов:

```
std::array<int, 4> myarray;
myarray = { 0, 1, 2, 3 }; // ок
myarray = { 8, 6 };  // ок, элементам 2 и 3 присвоен нуль!
myarray = { 0, 1, 3, 5, 7, 9 }; // нельзя, слишком много элементов в списке инициализаторов!
```

Доступ к значениям массива через оператор индекса осуществляется как обычно:

```
std::cout << myarray[1];
myarray[2] = 7;
```

Так же, как и в стандартных фиксированных массивах, оператор индекса не выполняет никаких проверок на диапазон. Если указан недопустимый индекс, то произойдут плохие вещи.

std::array поддерживает вторую форму доступа к элементам массива — **функция at()**, которая осуществляет проверку диапазона:

```
std::array<int, 4> myarray { 8, 6, 4, 1 };
myarray.at(1) = 7; // элемент массива под номером 1 - корректный, присваиваем ему значение 7
myarray.at(8) = 15; // элемент массива под номером 8 - некорректный, получим ошибку
```

В примере, приведенном выше, вызов `myarray.at(1)` проверяет, есть ли элемент массива под номером 1, и, поскольку он есть, возвращается **ссылка** на этот элемент. Затем мы присваиваем ему значение `7`. Однако, вызов `myarray.at(8)` не срабатывает, так как элемента под номером 8 в массиве нет. Вместо возвращения ссылки, функция at() выдает ошибку, которая завершает работу программы (на самом деле выбрасывается исключение типа `std::out_of_range`. Об исключениях мы поговорим на соответствующих уроках). Поскольку проверка диапазона выполняется, то функция at() работает медленнее (но безопаснее), чем оператор `[]`.

std::array автоматически делает все очистки после себя, когда выходит из области видимости, поэтому нет необходимости прописывать это вручную.

Размер и сортировка

С помощью **функции size()** можно узнать длину массива:

```
#include <iostream>
#include <array>

int main()
{
    std::array<double, 4> myarray{ 8.0, 6.4, 4.3, 1.9 };
    std::cout << "length: " << myarray.size();

    return 0;
}
```

Результат:

```
length: 4
```

Поскольку std::array не распадается в указатель при передаче в функцию, то функция size() будет работать, даже если её вызвать из другой функции:

```
#include <iostream>
#include <array>

void printLength(const std::array<double, 4> &myarray)
{
    std::cout << "length: " << myarray.size();
}

int main()
{
    std::array<double, 4> myarray { 8.0, 6.4, 4.3, 1.9 };

    printLength(myarray);

    return 0;
}
```

Результат тот же:

```
length: 4
```

Обратите внимание, Стандартная библиотека C++ использует термин «размер» для обозначения длины массива — не путайте это с результатами выполнения **оператора sizeof** с обычным фиксированным массивом, когда возвращается фактический размер массива в памяти (`размер_элемента * длина_массива`).

Также обратите внимание на то, что мы передаем std::array по (**константной**) ссылке. Это делается по соображениям производительности для того, чтобы компилятор не выполнял копирование массива при передаче в функцию.

Правило: Всегда передавайте std::array в функции по обычной или по константной ссылке.

Поскольку длина массива всегда известна, то **циклы foreach** также можно использовать с std::array:

```
std::array<int, 4> myarray { 8, 6, 4, 1 };

for (auto &element : myarray)
    std::cout << element << ' ';
```

Вы можете отсортировать std::array, используя **функцию std::sort()**, которая находится в заголовочном файле algorithm:

```
#include <iostream>
#include <array>
#include <algorithm> // для std::sort

int main()
{
    std::array<int, 5> myarray { 8, 4, 2, 7, 1 };
    std::sort(myarray.begin(), myarray.end()); // сортировка массива по возрастанию
//    std::sort(myarray.rbegin(), myarray.rend()); // сортировка массива по убыванию

    for (const auto &element : myarray)
        std::cout << element << ' ';

    return 0;
}
```

Результат:

```
1 2 4 7 8
```

Функция сортировки использует итераторы, которые мы еще не рассматривали. О них мы поговорим несколько позже.

Заключение

std::array — это отличная замена стандартных фиксированных массивов. Массивы, созданные с помощью std::array, более эффективны, так как используют меньше памяти. Единственными недостатками std::array по сравнению со стандартными фиксированными массивами являются немного неудобный синтаксис и то, что нужно явно указывать длину массива (компилятор не будет вычислять её за нас). Но это сравнительно незначительные нюансы. Рекомендуется использовать std::array вместо стандартных фиксированных массивов в любых нетривиальных задачах.